

Assignment 3 | Bridge Pattern

Merrey Kuatbay | SE 2537

Topic A - Shape Drawing Renderer

Repository: <https://github.com/bossmerej-ship-it/Assignment3-Bridge-Pattern>

Submitted source commit: 276269d14e29dc35d963890acc68e2d9ac83ce46

Design and UML

The two choices are shape and rendering format. Circle stores a radius and Square stores a side length. VectorRenderer, RasterRenderer, and AsciiRenderer provide the three formats. Shape receives a Renderer through its constructor and delegates rendering through that interface, so each shape can use each renderer without a subclass for every combination. This is the Bridge relationship described in [1].

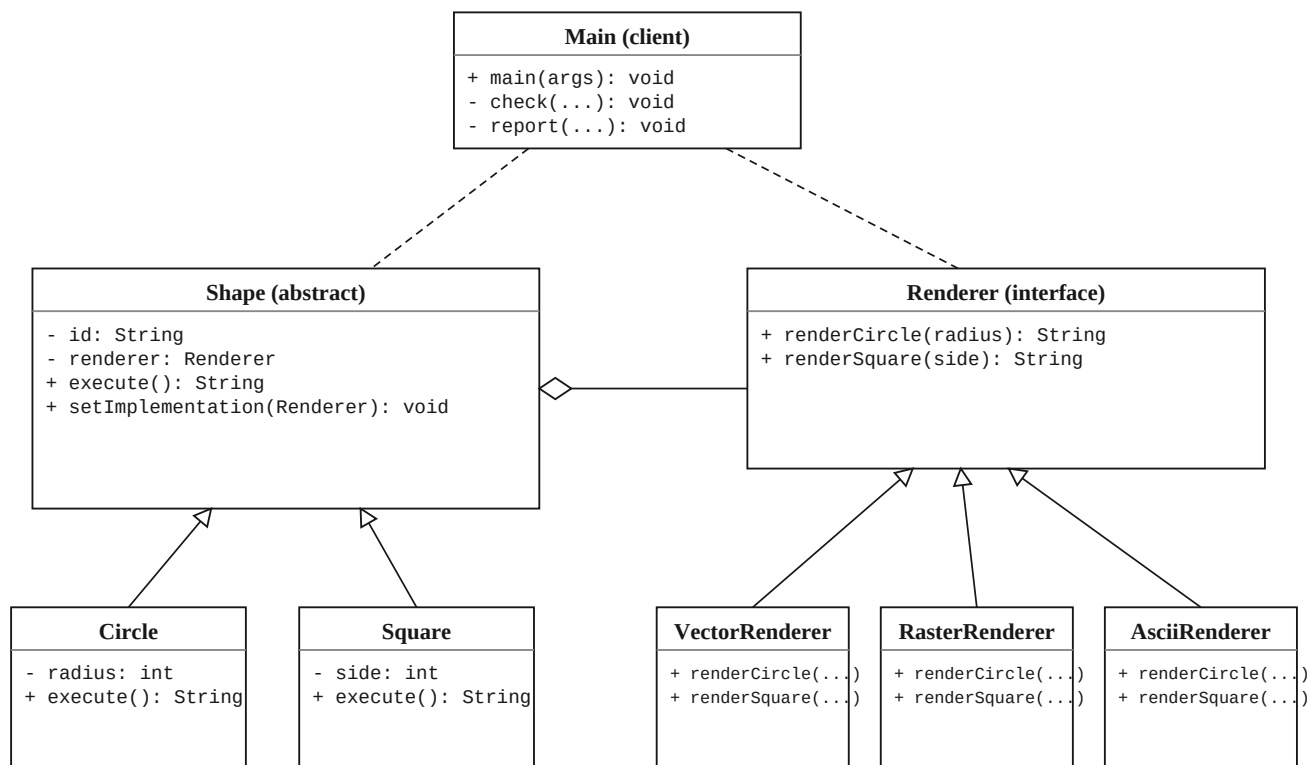


Figure 1. UML class diagram of the submitted code.

Code examples

Code example	Practice and benefit
<pre>Shape.java / Circle.java protected Renderer renderer; public abstract String execute(); public String execute() { return renderer.renderCircle(radius); }</pre>	Separation of responsibilities Shape stores the bridge reference and shared ID; Circle keeps shape behavior while the renderer produces its representation. Neither side constructs the other concrete class.
<pre>Main.java - T5 boolean sameObject = originalReference == afterReference; boolean stateUnchanged = originalId.equals(switchedShape.getId()) && originalRadius == switchedShape.getRadius();</pre>	Meaningful names sameObject checks reference identity. stateUnchanged names the ID and radius condition, making the runtime-switch evidence easy to read.
<pre>Shape.java public final void setImplementation(Renderer renderer) { this.renderer = Objects.requireNonNull(renderer, "renderer"); }</pre>	Small focused methods The setter replaces only the implementation reference. It does not reconstruct the shape or alter its domain data.
<pre>Main.java private static void check(...){ report(id, expected.equals(actual), ...); } private static void report(...){ total++; if (success) passed++; ... }</pre>	Avoiding duplicated workflow All seven checks use the same comparison and output helpers, rather than repeating PASS/FAIL logic for each renderer and shape.
<pre>Shape.java / Circle.java private final String id; private final int radius; public int getRadius() { return radius; }</pre>	Encapsulation of state ID and radius are private, final values. Narrow getters expose only what the client demonstration needs; renderer replacement cannot rewrite them.

Circle and Square pass their retained dimensions to the renderer. The shared check/report workflow keeps the demonstration compact while still calculating each result from the actual return value.

Demonstration and extension

T1-T7 compare actual results with expected strings. T5 also uses == to prove that one Circle object remains in use while its Renderer changes.

JDK 17 - build and demo

```
$ javac --release 17 -encoding UTF-8 -d out "@sources.txt"
$ java -cp out Main --demo
T1 PASS | Circle + VectorRenderer | result=VECTOR circle radius=2
T2 PASS | Circle + RasterRenderer | result=RASTER circle radius=2
T3 PASS | Square + VectorRenderer | result=VECTOR square side=3
T4 PASS | Square + RasterRenderer | result=RASTER square side=3
T5 PASS | sameObject=true | stateUnchanged=true | before=VECTOR circle radius=2 | after=RASTER
        circle radius=2
T6 PASS | Circle + AsciiRenderer | result=ASCII circle radius=2
T7 PASS | Square + AsciiRenderer | result=ASCII square side=3
SUMMARY: 7/7 PASS
```

Figure 2. Recorded output from the JDK 17 demo.

The first commit contained the Vector/Raster combinations and runtime replacement. The extension added AsciiRenderer and updated Main for T6-T7; Shape, Circle, Square, Renderer, VectorRenderer, and RasterRenderer stayed unchanged. extension.diff records the source changes.

Base I1/I2 commit: 30798a7b83e3050f9795afaf0ecacc3d05fd74b4

Extension changes within src/

```
A src/implementor/AsciiRenderer.java
M src/Main.java
```

Trade-off and Adapter

Bridge adds an interface and a delegation step, but it prevents shape-renderer combinations from multiplying as both dimensions grow. Adapter instead wraps an existing incompatible interface so it can be used through the interface a client expects. Here Renderer is designed as the bridge contract; it does not adapt a legacy API [1].

References

- [1] Gamma, E., Helm, R., Johnson, R., and Vlissides, J. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [2] Oracle. *Java Language Specification, Java SE 17 Edition*. <https://docs.oracle.com/javase/specs/jls/se17/jls17.pdf>
- [3] Bekenov, Y. *Assignment 3 | Bridge Pattern*, Astana IT University, 2026-2027. Supplied course instructions.